

Ex: Create authentication service that returns JWT

AuthenticationControl.java:

```
1 package com.cognizant.springlearn.controller;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import java.util.Base64;
6 import java.util.Date;
7
8 import io.jsonwebtoken.JwtBuilder;
9 import io.jsonwebtoken.Jwts;
10 import io.jsonwebtoken.SignatureAlgorithm;
11
12 import org.slf4j.Logger;
13 import org.slf4j.LoggerFactory;
14 import org.springframework.web.bind.annotation.GetMapping;
15 import org.springframework.web.bind.annotation.RequestHeader;
16 import org.springframework.web.bind.annotation.RestController;
17
18 @RestController
19 public class AuthenticationController {
20
21     private static final Logger LOGGER =
22         LoggerFactory.getLogger(AuthenticationController.class);
23
24     @GetMapping("/authenticate")
25     public Map<String, String> authenticate(
26         @RequestHeader("Authorization") String authHeader) {
27
28         LOGGER.info("START");
29
30         LOGGER.debug("Authorization Header : {}", authHeader);
31
32         Map<String, String> map = new HashMap<>();
33
34         String user = getUser(authHeader);
```

```

36     LOGGER.debug("Authenticated User : {}", user);
37
38     String token = generateJwt(user);
39
40     map.put("token", token);
41
42     LOGGER.info("END");
43
44     return map;
45 }
46
47 private String getUser(String authHeader) {
48
49     LOGGER.debug("Inside getUser()");
50
51     // Remove "Basic "
52     String encodedCredentials = authHeader.substring(6);
53
54     LOGGER.debug("Encoded Credentials : {}", encodedCredentials);
55
56     byte[] decodedBytes = Base64.getDecoder().decode(encodedCredentials);
57
58     String credentials = new String(decodedBytes);
59
60     LOGGER.debug("Decoded Credentials : {}", credentials);
61
62     String user = credentials.substring(0, credentials.indexOf(":"));
63
64     LOGGER.debug("User : {}", user);
65
66     return user;
67 }
68
69 private String generateJwt(String user) {
70
71     LOGGER.debug("Inside generateJwt()");
72
73     JwtBuilder builder = Jwts.builder();
74
75     builder.setSubject(user);
76
77     // Issue time
78     builder.setIssuedAt(new Date());
79
80     // Expiry after 20 minutes
81     builder.setExpiration(new Date(System.currentTimeMillis() + 1200000));
82
83     builder.signWith(SignatureAlgorithm.HS256, "secretkey");
84
85     String token = builder.compact();
86
87     LOGGER.debug("Generated Token : {}", token);
88
89     return token;
90 }
91 }

```

SecurityConfig.java:

```
1 package com.cognizant.springlearn.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.Customizer;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.core.userdetails.User;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
10 import org.springframework.security.crypto.password.PasswordEncoder;
11 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
12 import org.springframework.security.web.SecurityFilterChain;
13
14 @Configuration
15 public class SecurityConfig {
16
17     @Bean
18     public InMemoryUserDetailsManager userDetailsService(PasswordEncoder passwordEncoder) {
19
20         UserDetails user = User.withUsername("user")
21             .password(passwordEncoder.encode("pwd"))
22             .roles("USER")
23             .build();
24
25         UserDetails admin = User.withUsername("admin")
26             .password(passwordEncoder.encode("pwd"))
27             .roles("ADMIN")
28             .build();
29
30         return new InMemoryUserDetailsManager(user, admin);
31     }
32
33     @Bean
34     public PasswordEncoder passwordEncoder() {
35         return new BCryptPasswordEncoder();
36     }
37
38     @Bean
39     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
40
41         http
42             .csrf(csrf -> csrf.disable())
43             .httpBasic(Customizer.withDefaults())
44             .authorizeHttpRequests(auth -> auth
45                 .requestMatchers("/countries/**").hasRole("USER")
46                 .requestMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
47                 .anyRequest().permitAll());
48
49         return http.build();
50     }
51 }
```

Output:

```
C:\Users\anjan>curl -s -u user:pwd http://localhost:8090/authenticate
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VybmVhWF0IjoxNzgzOTU2MzQ0LCJleHAiOjE3ODM5NTc1NDk5LjYRCHVod2oJ3t4717fEkbaAI
sR-RTnxn22AtBdUuekM"}

```